

Experiencia de Práctica N.º 02

Teoría de Conjuntos en Python — Ejercicios de programación

Universidad Católica de Santa María · Escuela Profesional de Ingeniería de Sistemas

Metodología de la Programación / Matemática — Semestre I 2026

Duración estimada: 60 minutos

Indicaciones generales. Resuelve cada actividad en Python (.py o notebook). Usa `print()` para mostrar cada resultado solicitado y verifica tus respuestas antes de pasar a la siguiente actividad. Los ejercicios retoman las convenciones de la Experiencia de Práctica N.º 01 (variable `a`, índices, longitud `n = len(a)`) y la sesión de Teoría de Conjuntos.

Actividad 1 · Representación y pertenencia en Python

(10 min)

Ejercicio 1.1 (conjuntos por comprensión). Considera el conjunto $A = \{x \mid x \in \mathbb{N}, 1 \leq x \leq 30, x \equiv 0 \pmod{3}\}$.

- Constrúyelo en Python usando comprensión de conjuntos (*set comprehension*).
- Verifica con `in` / `not in` si $9 \in A$, $10 \in A$ y $27 \in A$.
- Calcula $|A|$ con `len(A)` y compáralo con el valor que esperarías analíticamente.
- Construye $B = \{x \mid x \in A, x > 15\}$ a partir de `A` (no de `range` otra vez) y verifica si $B \subseteq A$ usando `B.issubset(A)`.

```
# Plantilla: completa las líneas marcadas con TODO
A = { }                                # TODO: comprension de conjuntos
print(9 in A, 10 in A, 27 in A)
print(len(A))
B = { }                                # TODO: a partir de A
print(B.issubset(A))
```

Actividad 2 · Operaciones de conjuntos “a mano”

(15 min)

Ejercicio 2.1. Sin usar los operadores `|` & `-` `^` ni los métodos `union/intersection/difference`, implementa tus propias funciones recorriendo los conjuntos con `for`.

- `union_manual(A, B)`
- `interseccion_manual(A, B)`
- `diferencia_manual(A, B)` (elementos de `A` que no están en `B`)
- Compara cada resultado con el operador nativo (`assert union_manual(A,B) == A | B`) usando $A = \{1, 2, 3, 4\}$, $B = \{3, 4, 5, 6\}$.

```
def union_manual(A, B):
    resultado = set()
    for elem in A:
        resultado.add(elem)
    # TODO: agregar tambien los elementos de B
    return resultado

def interseccion_manual(A, B):
    resultado = set()
    for elem in A:
        if elem in B:
            resultado.add(elem)
    return resultado

def diferencia_manual(A, B):
    # TODO
    pass

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
assert union_manual(A, B) == A | B
assert interseccion_manual(A, B) == A & B
assert diferencia_manual(A, B) == A - B
print("Todas las pruebas pasaron")
```

Actividad 3 · Conjunto potencia programático

(10 min)

Ejercicio 3.1. Sea $A = \{1, 2, 3\}$.

- Escribe una función `powerset(A)` que devuelva $P(A)$ como una lista de conjuntos, usando `itertools.combinations` para cada tamaño $k = 0, \dots, |A|$.
- Verifica que `len(powerset(A)) == 2**len(A)`.
- Imprime $P(A)$ y confirma manualmente que aparecen $\emptyset$ y A .
- Reto opcional:** escribe una segunda versión `powerset_recursivo(A)` sin usar `itertools`, solo recursión.

```
from itertools import combinations

def powerset(A):
    A = list(A)
    resultado = []
    for k in range(len(A) + 1):
        for combo in combinations(A, k):
            resultado.append(set(combo))
    return resultado

A = {1, 2, 3}
P = powerset(A)
print(len(P) == 2 ** len(A))
for s in P:
    print(s)
```

Actividad 4 · Relaciones binarias como conjuntos de tuplas**(15 min)**

Ejercicio 4.1. Sea $A = \{1, 2, 3, 4, 5, 6\}$ y la relación $R = \{(a, b) \mid a, b \in A, a \text{ es múltiplo de } b\}$.

- Construye R en Python con comprensión de conjuntos.
- Reutiliza (o vuelve a escribir) las funciones `es_reflexiva`, `es_simetrica`, `es_antisimetrica` y `es_transitiva` vistas en la sesión teórica y aplícalas a R .
- Indica cuáles de las cuatro propiedades cumple R e imprime el resultado de forma legible, por ejemplo: `R es reflexiva: True`.

```
A = {1, 2, 3, 4, 5, 6}
R = {(a, b) for a in A for b in A if a % b == 0}

def es_reflexiva(R, A):
    return all((a, a) in R for a in A)

def es_simetrica(R):
    return all((b, a) in R for (a, b) in R)

def es_antisimetrica(R):
    return all(a == b for (a, b) in R if (b, a) in R)

def es_transitiva(R):
    return all((a, c) in R for (a, b1) in R for (b2, c) in R if b1 == b2)

print("Reflexiva:", es_reflexiva(R, A))
print("Simetrica:", es_simetrica(R))
print("Antisimetrica:", es_antisimetrica(R))
print("Transitiva:", es_transitiva(R))
```

Ejercicio 4.2 (orden parcial e inversa).

- Con las funciones anteriores, escribe `es_orden_parcial(R, A)` y verifica si la relación del ejercicio 4.1 lo es.
- Escribe `relacion_inversa(R)` y calcula R^{-1} para la misma relación. ¿ R^{-1} representa “es múltiplo de” o “es divisor de”? Justifica en un comentario dentro del código.

```
def es_orden_parcial(R, A):
    return es_reflexiva(R, A) and es_antisimetrica(R) and es_transitiva(R)

def relacion_inversa(R):
    return {(b, a) for (a, b) in R}

print("Orden parcial:", es_orden_parcial(R, A))
R_inv = relacion_inversa(R)
# TODO: comentario explicando que representa R_inv
print(R_inv)
```

Actividad 5 · Integrador de cierre**(10 min)**

Ejercicio 5.1. Retoma la sucesión de la Experiencia de Práctica N.º 01: `a = ['Carlo', 'Jose', 'Luis']`.

- Construye el conjunto de índices válidos $I = \{0, 1, \dots, n-1\}$ con $n = \text{len}(a)$, usando comprensión de conjuntos (no lo escribas a mano).
- Define la relación R sobre a como los pares $(\text{palabra}_i, \text{palabra}_j)$ tales que $\text{len}(\text{palabra}_i) \leq \text{len}(\text{palabra}_j)$.
- Aplica `es_reflexiva`, `es_antisimetrica` y `es_transitiva` a esta nueva relación.
- Conjetura en un comentario: ¿ R es un orden parcial? ¿Cambiaría la respuesta si dos palabras tuvieran exactamente la misma longitud?

```
a = ['Carlo', 'Jose', 'Luis']
n = len(a)
I = {i for i in range(n)}          # TODO: usar I si lo necesitas

R = {(x, y) for x in a for y in a if len(x) <= len(y)}
print(R)

print("Reflexiva:", es_reflexiva(R, set(a)))
print("Antisimetrica:", es_antisimetrica(R))
print("Transitiva:", es_transitiva(R))
# TODO: comentario con tu conjetura
```

Reto opcional (si sobra tiempo): repite el ejercicio 5.1 usando tu propio nombre completo en lugar de 'Carlo Jose Luis' y compara si la conjetura se mantiene.

Referencias: Jiménez Murillo (2015) *Matemáticas para la computación*; Alfaomega, *Matemáticas discretas: aplicaciones y ejercicios*; Sílabo 7101245 — Metodología de la Programación, EPIS UCSM, 2026.